

Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

Inode

```
geya@LenovoL480:~/OS/Practical/Week 10$ echo "Hello, this is an inode test." > original.txt
geya@LenovoL480:~/OS/Practical/Week 10$ ls -li original.txt
41392 original.txt
```

41392- Inode number

Stat

```
geya@LenovoL480:~/OS/Practical/Week 10$ stat original.txt
  File: original.txt
  size: 30          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 41392         Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/   geya)   Gid: ( 1000/   geya)
Access: 2026-09-23 20:15:20.968197387 +0000
Modify: 2026-09-23 20:15:20.968197387 +0000
Change: 2026-09-23 20:15:20.968197387 +0000
 Birth: 2026-09-23 20:15:20.968197387 +0000
geya@LenovoL480:~/OS/Practical/Week 10$ |
```

Find

```
geya@LenovoL480:~/OS/Practical/Week 10$ find . -inum 41392
./hardlink.txt
./original.txt
geya@LenovoL480:~/OS/Practical/Week 10$ |
```

Hard link

```
geya@LenovoL480:~/OS/Practical/Week 10$ ln original.txt hardlink.txt
geya@LenovoL480:~/OS/Practical/Week 10$ ls -li original.txt hardlink.txt
41392 -rw-r--r-- 2 geya geya 30 Sep 23 20:15 hardlink.txt
41392 -rw-r--r-- 2 geya geya 30 Sep 23 20:15 original.txt
geya@LenovoL480:~/OS/Practical/Week 10$ |
```

Symbolic link

```
geya@LenovoL480:~/OS/Practical/Week 10$ ln -s original.txt symlink.txt
geya@LenovoL480:~/OS/Practical/Week 10$ ls -li original.txt hardlink.txt symlink.txt
41392 -rw-r--r-- 2 geya geya 30 Sep 23 20:15 hardlink.txt
41392 -rw-r--r-- 2 geya geya 30 Sep 23 20:15 original.txt
41393 lrwxrwxrwx 1 geya geya 12 Sep 23 20:18 symlink.txt -> original.txt
geya@LenovoL480:~/OS/Practical/Week 10$ |
```

Symbolic link has its own Inode whereas hard link shares inode with original file. Symbolic link points to Original so if it's deleted it will give error.

```
geya@LenovoL480:~/OS/Practical/Week 10$ rm original.txt
geya@LenovoL480:~/OS/Practical/Week 10$ cat hardlink.txt
Hello, this is an inode test.
geya@LenovoL480:~/OS/Practical/Week 10$ cat symlink.txt
cat: symlink.txt: No such file or directory
```

Task 2

Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and Implementation complexity.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>
#include <time.h>

/* =====
   TRADITIONAL READ/WRITE
   ===== */

void traditional_copy(char *source, char *destination)
{
    int src, dest;
    char buffer[1024];
    int bytes;

    src = open(source, O_RDONLY);

    if (src == -1)
    {
        perror("Source file");
        return;
    }
}
```

```
dest = open(destination,
              O_WRONLY | O_CREAT | O_TRUNC,
              0644);

if (dest == -1)
{
    perror("Destination file");
    close(src);
    return;
}

while ((bytes = read(src, buffer, sizeof(buffer))) > 0)
{
    write(dest, buffer, bytes);
}

close(src);
close(dest);
}

/* =====
MEMORY MAPPED COPY
| ===== */
```

```
void mmap_copy(char *source, char *destination)
{
    int src, dest;
    struct stat file_info;
    size_t size;

    /* Open source */
    src = open(source, O_RDONLY);

    if (src == -1)
    {
        perror("Source file");
        return;
    }

    /* Get file size */
    if (fstat(src, &file_info) == -1)
    {
        perror("fstat");
        close(src);
        return;
    }

    size = file_info.st_size;
}
```

```
if (size == 0)
{
    printf("File is empty.\n");
    close(src);
    return;
}

/* Map source file into memory */

char *source_data = mmap(NULL,
                          size,
                          PROT_READ,
                          MAP_PRIVATE,
                          src,
                          0);

if (source_data == MAP_FAILED)
{
    perror("mmap");
    close(src);
    return;
}

| /* Create destination file */
```

```
dest = open(destination,  
             O_RDWR | O_CREAT | O_TRUNC,  
             0644);  
  
if (dest == -1)  
{  
    perror("Destination file");  
    munmap(source_data, size);  
    close(src);  
    return;  
}  
  
/* Set destination file size */  
  
if (ftruncate(dest, size) == -1)  
{  
    perror("ftruncate");  
  
    munmap(source_data, size);  
    close(src);  
    close(dest);  
  
    return;  
}
```

```
/* Map destination file */

char *destination_data = mmap(NULL,
                               size,
                               PROT_READ | PROT_WRITE,
                               MAP_SHARED,
                               dest,
                               0);

if (destination_data == MAP_FAILED)
{
    perror("mmap destination");

    munmap(source_data, size);
    close(src);
    close(dest);

    return;
}

/* Copy data */

memcpy(destination_data, source_data, size);
```

```

    /* Write changes to file */

    msync(destination_data, size, MS_SYNC);

    /* Remove mappings */

    munmap(source_data, size);
    munmap(destination_data, size);

    close(src);
    close(dest);
}

/* =====
MAIN
===== */

int main()
{
    char source[100];

    printf("Enter source file: ");
    scanf("%99s", source);

```

```

/* =====
   TRADITIONAL COPY
   ===== */

printf("\n===== TRADITIONAL I/O =====\n");

clock_t start = clock();

traditional_copy(source, "traditional_copy.txt");

clock_t end = clock();

double traditional_time =
    (double)(end - start) / CLOCKS_PER_SEC;

printf("Traditional read/write completed.\n");
printf("Time: %f seconds\n", traditional_time);

/* =====
   MMAP COPY
   ===== */

printf("\n===== MEMORY MAPPED I/O =====\n");

start = clock();

```

```

mmap_copy(source, "mmap_copy.txt");

end = clock();

double mmap_time =
    (double)(end - start) / CLOCKS_PER_SEC;

printf("Memory-mapped copy completed.\n");
printf("Time: %f seconds\n", mmap_time);

/* =====
COMPARISON
===== */

printf("\n===== COMPARISON =====\n");

printf("Traditional I/O : read() + write()\n");
printf("Memory mapped   : mmap() + memcpy()\n")

printf("Traditional time: %f seconds\n",
        traditional_time);

printf("mmap time       : %f seconds\n",
        mmap_time);

printf("\nBoth copies completed.\n");

return 0;
}

```

Output:

```
geya@LenovoL480:~/OS/Practical/Week 10$ ./task2
Enter source file: input.txt
```

```
===== TRADITIONAL I/O =====
```

```
Traditional read/write completed.
```

```
Time: 0.000259 seconds
```

```
===== MEMORY MAPPED I/O =====
```

```
Memory-mapped copy completed.
```

```
Time: 0.000540 seconds
```

```
===== COMPARISON =====
```

```
Traditional I/O : read() + write()
```

```
Memory mapped   : mmap() + memcpy()
```

```
Traditional time: 0.000259 seconds
```

```
mmap time       : 0.000540 seconds
```

```
Both copies completed.
```

```
geya@LenovoL480:~/OS/Practical/Week 10$ |
```